

GenAI, ML, LLM, RAG & Agentic AI

A detailed guide to how these terms fit together

The big picture

Artificial intelligence (AI) is the broad goal of building machines that can do tasks normally requiring human intelligence, such as understanding language, recognizing images, making decisions, or solving problems. The five terms in this guide are not competing ideas. They are layers, each a more specific piece inside the one before it. Seeing the order makes the field much less confusing.

Machine Learning (ML)

Traditional software follows rules a programmer writes by hand: if this happens, do that. Machine learning flips the approach. You give the system a large amount of data, and it discovers the patterns itself. A spam filter is a classic example. Instead of someone listing every suspicious phrase, the model studies thousands of emails already labeled “spam” or “not spam” and learns what separates them. Once trained, it can judge new emails it has never seen.

Most of the AI you meet daily is ML: product recommendations, credit card fraud detection, voice assistants, and traffic predictions on maps. These systems typically classify things (“is this fraud?”) or predict numbers (“how long will this trip take?”).

Generative AI (GenAI)

Generative AI is a branch of ML focused on **creating** new content rather than just labeling or predicting. A classic ML model might look at a photo and tell you it contains a cat. A generative model can produce a brand-new picture of a cat, write a story about it, or compose a jingle. It works across text, images, audio, video, and software code.

This works because these models learn the underlying structure of their training data so well that they can produce fresh examples that follow the same patterns. The recent surge of interest in AI, from image generators to chat assistants, is almost entirely due to GenAI.

Large Language Models (LLMs)

An LLM is a generative model specialized for language. It is trained on enormous collections of text, including books, websites, articles, and code, with a deceptively simple goal: predict the next piece of text (called a token) in a sequence. To do that well across billions of examples, the model has to absorb grammar, facts, writing styles, reasoning patterns, and programming conventions. That is why a single LLM can summarize a contract, translate a paragraph, explain a math concept, and debug a script. Claude, GPT, and Gemini are all LLMs.

LLMs are powerful, but they have real limitations:

- **Knowledge cutoff.** They only know what was in their training data, so they can be out of date.
- **No access to private data.** They can't see your company documents or personal files unless you supply them.
- **Hallucination.** They can state something false in a fluent, confident tone, because they generate plausible text rather than look up verified facts.

- **Limited working memory.** They can only consider a certain amount of text at once.

Much of the recent innovation in AI is about working around these limits, and RAG and agents are two of the most important answers.

Retrieval-Augmented Generation (RAG)

RAG is a technique that gives an LLM access to knowledge it wasn't trained on. Instead of asking the model to answer purely from memory, the system first **retrieves** relevant information and includes it in the prompt, so the model answers with real material in front of it. Think of the difference between a closed-book exam and an open-book one.

A typical RAG pipeline works like this:

1. **Prepare.** Your documents (manuals, policies, wikis, PDFs) are split into small chunks and converted into numerical representations called embeddings, which capture meaning. These are stored in a vector database.
2. **Retrieve.** When a user asks a question, it is converted the same way, and the database returns the chunks whose meaning is closest.
3. **Generate.** Those chunks are placed into the prompt along with the question, and the LLM writes an answer grounded in them.

The benefits are substantial. Answers can reflect current or private information, hallucinations decrease because the model is anchored to real text, and the system can cite its sources so people can verify them. It's also far cheaper and faster than retraining a model whenever your information changes. A support chatbot that answers from your latest product documentation is a typical example. RAG's quality depends heavily on retrieval, though: if the wrong chunks are found, the answer suffers.

Agentic AI

A standard chatbot answers a question and stops. Agentic AI is about systems that pursue a goal. An agent, powered by an LLM, can break a goal into steps, choose and use tools, observe the results, and adjust its plan until the task is complete. Tools might include web search, running code, calling APIs, reading and editing files, querying databases, or sending messages.

Suppose you ask, "Research three competitors and put a comparison in a spreadsheet." An agent might search the web, open several pages, extract pricing and features, notice that one page is missing data and try another source, write everything into a spreadsheet, and then check its own work. That loop of planning, acting, observing, and correcting is what makes it "agentic."

RAG is frequently one of the tools an agent uses, for example to look up internal documents. Agents can also work in teams, with different agents handling research, writing, and review.

The trade-off is risk. Because agents take actions, mistakes can compound or have real consequences, like sending the wrong email or deleting a file. Good agent design includes permission limits, logging, and human approval for sensitive steps.

How it all fits together

Term	What it is	Relationship
AI	Machines doing tasks that need intelligence	The umbrella
ML	AI that learns from data	Inside AI
GenAI	ML that creates new content	Inside ML
LLM	GenAI models for language	Inside GenAI
RAG	An LLM plus retrieved knowledge	Technique built on LLMs
Agentic AI	An LLM plus tools, planning, and autonomy	System built on LLMs

Where to start

If you're learning this, follow the same order: understand how LLMs behave and how to prompt them, build a simple RAG app over your own documents, and then add tools so it can take actions.